Kubernetes Device Plugin Config Validator

Project architecture, technology selection, application workflow, and Git and GitHub fundamentals

Project ID	G401
Prepared by	Mayank Singhal
Track	Open Source Developer OJT · 4 weeks
Stack	Python 3.10+, PyYAML, argparse, pytest, GitHub Actions

Technical document · Version 2.2

October 2026

Contents

1 **Project Overview**
Background · problem statement · purpose and objectives · key features · intended users · requirements · scope

2 **Technology Stack**
Stack at a glance · selected technologies · development environment · why no database · alternatives compared

3 **Basic Project Architecture**
Architectural style · design principles · folder structure · responsibilities · component contracts · interactions · data model · path model · rule catalog · error handling · security · performance

4 **Project Workflow**
Flowchart · inputs · processing steps · data transformation walkthrough · outputs and exit codes · scenario matrix

5 **Git and GitHub Documentation**
Git vs GitHub · repositories, commits, branches, merges · README and .gitignore · everyday operations · issues and pull requests · collaboration · how this project uses Git

6 **Technical Justification and Conclusion**
Decision record · quality attributes · maintainability · scalability · future enhancements · risks · roadmap · conclusion

A **Appendices**
Glossary · Git command cheat sheet · references

01 Project Overview

What the project is, the problem it solves, who it is for, and exactly what it must and must not do.

1.1 Background: Kubernetes and device plugins

Kubernetes is an open-source system that runs applications across a group of machines called a *cluster*. Each machine is a **node**, and an agent on every node, the **kubelet**, starts and supervises the application containers placed on it.

Some nodes contain specialised hardware: **GPUs** for graphics and AI, **FPGAs** (re-programmable chips), high-speed or SR-IOV network cards, and AI/ML accelerators. Kubernetes cannot discover or share this hardware on its own. Instead, it provides a **device plugin framework**: a vendor ships a small program, the **device plugin**, that runs on each node (usually as a DaemonSet), registers with the kubelet over a local gRPC socket, reports which devices exist, and allocates them to applications that request them, for example `nvidia.com/gpu: 1`.

Every device plugin is controlled by a **configuration file**, written in JSON or YAML and usually delivered to the plugin through a Kubernetes ConfigMap. The file decides which devices are discovered, how they are shared, which flags are enabled and what limits apply.

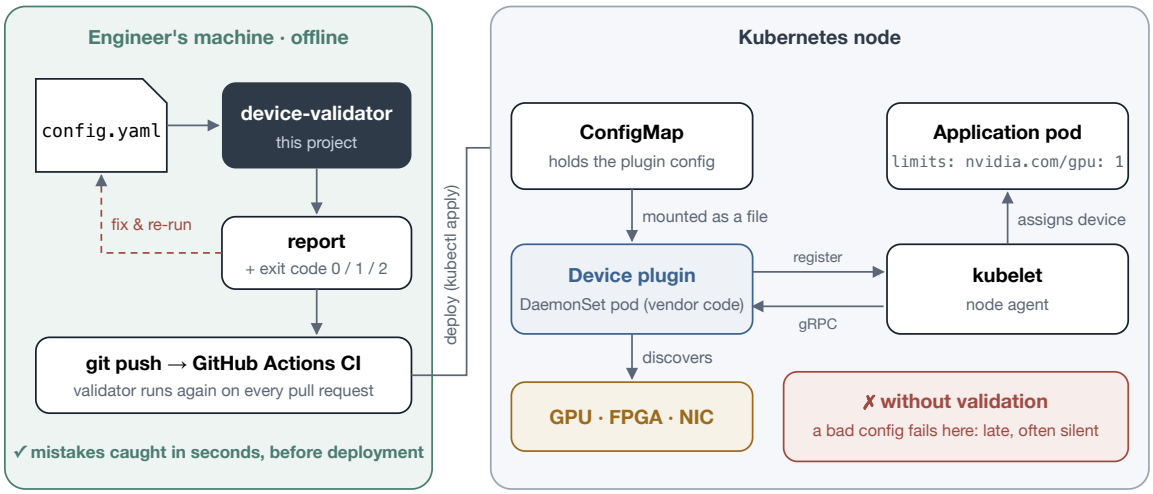


Figure 1. Where the validator fits. The device plugin reads its config on the node; the validator checks the same file much earlier, on the engineer's machine and in CI.

1.2 Problem statement

Device plugin configuration files are deeply nested, and small mistakes are easy to make and hard to see. The table below lists the kinds of mistakes that occur and what typically happens when they reach a node.

Mistake type	Example	Typical consequence on the node
Broken syntax	`devices:` followed by badly indented lines	The plugin cannot parse the file and exits; the DaemonSet pod enters a restart loop.
Missing required key	A device entry without `name`	Devices are silently skipped, so applications wait forever for hardware that "doesn't exist".
Wrong data type	`replicas: "four"` or `replicas: true`	The value is rejected or misread; behaviour differs from what the author intended.
Out-of-range value	`memory.limit: -5`, `replicas: 0`	Plugin start-up aborts, or the node advertises nonsense capacity to the scheduler.
YAML type surprise	`enabled: yes` becomes `True`; `version: 1.0` becomes a float	A value that looks right in the editor is a different type after parsing.

Today these problems are usually found *after* deployment, by reading node logs, restarting containers and inspecting DaemonSets, often with root access to the node. That is slow, needs a live cluster, and can disrupt running workloads. Existing tools such as **kubeval**, **kubeconform**, **kube-linter** and **Monokle** validate Kubernetes *manifests* (Pods, ConfigMaps, Deployments), but none of them understands the plugin-specific settings *inside* those files.

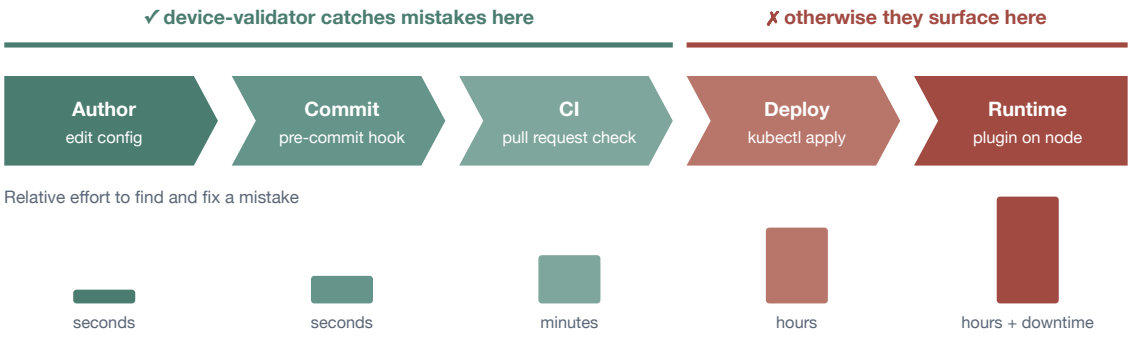


Figure 2. "Shift left": the earlier a mistake is caught, the cheaper it is to fix. The validator moves detection to the first three stages.

> **PROBLEM IN ONE SENTENCE** Engineers have no lightweight, offline way to check device plugin configuration files before deploying them, so mistakes are discovered late, on live nodes, where they are slow and risky to diagnose.

1.3 Purpose and objectives

The purpose of the project is to build a **standalone Python command-line tool** that safely reads local device plugin configuration files, walks through their nested structure, evaluates every field against a documented, mentor-approved rule catalog, and produces a clear diagnostic report. Each reported problem shows the exact field path, rule ID, severity and a fix hint, without revealing sensitive values and without needing a cluster or real hardware.

#	Objective	Success metric	How it is measured
1	Catch mistakes before deployment	100% agreement with expected findings	Benchmark fixtures with known, planted problems
2	Point to the exact location	Every finding has a precise field path	Integration tests assert path strings exactly
3	Never crash on bad input	0 unhandled exceptions	Malformed, empty, scalar, huge and deeply nested fixtures
4	Never leak secrets	0 secret strings in any output	Fixture full of fake secrets; output is searched for each one
5	Be fast enough for hooks and CI	< 200 ms per file, < 30 MB memory	Timed test on a 500-line nested file
6	Be trustworthy code	≥ 85% test coverage	coverage.py enforced in GitHub Actions
7	Fit into automation	Standard exit codes 0 / 1 / 2	End-to-end tests on every outcome

1.4 Key features

1. CLI interface
One command, `device-validator check --file config.yaml`, with clear help text, verbose mode and informative usage errors (built with `argparse`).

2. Safe parsing engine
Reads JSON and YAML with safe loaders only. Syntax errors, empty files and unsupported top-level types become clear findings, never crashes.

3. Nested field path traverser
Walks every key and list item depth-first and records its full address, e.g. `spec.devices[0].memory.limit`.

4. Deterministic rule catalog
An extensible, documented set of rules: required keys, data types (string, integer, boolean) and numeric ranges.

5. Structured findings
Every problem is a record with rule ID, field path, severity, message, a safe observed value and a remediation hint.

6. Sensitive value masking
Passwords, tokens, keys and certificates are replaced with `***REDACTED***` before any output is produced.

7. Readable terminal report
Colour-coded findings, fix hints and a summary of rules evaluated, errors and warnings, plus POSIX exit codes.

8. Sample fixture suite
Valid, partially valid and deliberately corrupted configuration files that prove the tool works and guard against regressions.

Stretch goals (if time permits)

JSON output
`--format=json` for CI tools and dashboards.

Strict mode
`--strict` treats warnings as failures (exit 1).

Directory validation
Validate every config file in a folder in one run.

External rule profiles
Load vendor rule sets from a YAML/JSON file.

Automated CI quality gate
GitHub Actions workflow that fails the build if tests fail or coverage drops below 85%.

1.5 Intended users

PRIMARY USER

Platform / infrastructure engineer

Writes and maintains device plugin configs for GPU and accelerator nodes.

- **Needs:** confidence a config is correct before rollout.
- **Gets:** exact field paths and fix hints in seconds.

PRIMARY USER

DevOps / CI pipeline owner

Owns the checks that guard the main branch of an infrastructure repository.

- **Needs:** a reliable, scriptable pass/fail signal.
- **Gets:** exit codes, strict mode and JSON output.

SECONDARY USER

Device plugin developer / contributor

Edits sample configurations and documentation in plugin repositories.

- **Needs:** fast feedback while editing examples.
- **Gets:** a local check with no cluster required.

SECONDARY USER

Student / Kubernetes newcomer

Learning how device plugins and their configuration work.

- **Needs:** to understand *why* a config is wrong.
- **Gets:** plain-language messages and remediation hints.

User stories

ID	As a…	I want to…	So that…
US-1	platform engineer	validate a config file with one command	I find mistakes before they reach a node
US-2	platform engineer	see the exact path and a fix hint for each problem	I can correct it without guessing
US-3	CI owner	get a non-zero exit code when errors exist	the pipeline blocks the merge automatically
US-4	CI owner	treat warnings as failures on demand	I can enforce stricter standards on main
US-5	security reviewer	be sure the output never contains secrets	CI logs can be shared safely
US-6	contributor	validate a whole folder of samples	every example in the repository stays correct

1.6 Requirements

Functional requirements

ID	Requirement	Priority
FR-1	Accept one or more file paths through a `check` command; show help with `--help`.	Must
FR-2	Verify the file exists, is a regular file, is readable and is below a size limit.	Must
FR-3	Parse JSON with `json` and YAML with PyYAML's safe loader; report syntax errors with line and column.	Must
FR-4	Reject empty documents and top-level values that are not mappings.	Must
FR-5	Traverse all nested dictionaries and lists, tracking each node's field path.	Must
FR-6	Evaluate required-key, type and range rules from the active rule profile.	Must
FR-7	Produce a finding for every violation with rule ID, path, severity, message, observed value and hint.	Must
FR-8	Mask sensitive values in all output.	Must
FR-9	Print a human-readable report and summary; exit with 0, 1 or 2.	Must

ID	Requirement	Priority
FR-10	Offer `--format=json`, `--strict`, directory input and external rule files.	Stretch

Non-functional requirements

ID	Quality	Requirement
NFR-1	**Reliability**	No unhandled exception for any input file, however malformed.
NFR-2	**Security**	Never execute content from input files; never print secret values or raw parser snippets.
NFR-3	**Performance**	Validate a 500-line nested file in under 200 ms using under 30 MB of memory.
NFR-4	**Determinism**	Identical input always gives identical output, in the same order.
NFR-5	**Portability**	Runs on Linux, macOS and Windows with Python 3.10 or newer; one runtime dependency.
NFR-6	**Maintainability**	≥ 85% test coverage; modules with a single responsibility; documented rule catalog.
NFR-7	**Usability**	Messages are actionable: each finding says what is wrong, where, and how to fix it.

1.7 Scope boundaries

In scope

- Local JSON and YAML configuration files
- Static checks: syntax, required keys, types, ranges
- Documented, mentor-approved rule catalog
- Terminal report, exit codes, optional JSON output
- Secret masking in every output
- Automated tests and CI on GitHub

Out of scope

- Connecting to a live Kubernetes cluster
- Talking to kubelet or device plugin gRPC sockets
- Accessing physical GPUs, FPGAs or network cards
- Certifying hardware compatibility or health
- Editing or auto-fixing configuration files
- Validating ordinary Kubernetes manifests (other tools do this)

02 Technology Stack

The languages, libraries and tools the project uses, why each one was chosen, and how they compare with the alternatives.

2.1 Stack at a glance

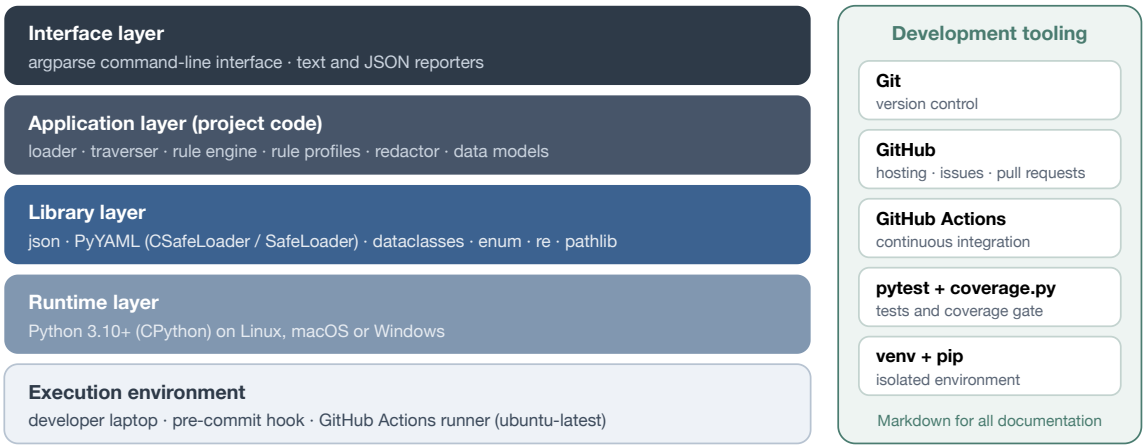


Figure 3. The technology stack as layers. Each layer only depends on the layers beneath it; tooling supports development but is not needed to run the tool.

2.2 Selected technologies

Technology	Version / licence	Role in the project	Why it was selected
Python	3.10+ · PSF	Programming language and runtime	Readable and widely known; nested dictionaries and lists map directly onto JSON and YAML; pre-installed on most Linux servers and CI runners. 3.10 brings modern type-hint syntax (`str | None`) and structural pattern matching.
argparse	stdlib	Command-line interface	Ships with Python, so no dependency. Supports subcommands (`check` , `rules`), typed options, automatic `--help` , and exits with code 2 on bad arguments, matching our "could not start" code.
json	stdlib	JSON parsing	Safe by design (produces only plain data). `JSONDecodeError` provides line and column without echoing file content.
PyYAML	6.x · MIT	YAML parsing	The de-facto YAML library for Python. `SafeLoader` builds only standard types and can never instantiate arbitrary objects. `CSafeLoader` (libyaml, written in C) is several times faster and is used automatically when installed.
dataclasses, enum	stdlib	Data models	Concise, typed record classes (`Rule` , `Finding` , `Report`) and a fixed `Severity` set; `frozen=True` makes findings immutable.
re, pathlib	stdlib	Redaction, file handling	Regular expressions detect secret-looking values; `pathlib` gives safe, cross-platform file paths.
pytest	8.x · MIT	Test framework	Plain `assert` statements, fixtures for sample files, and `@pytest.mark.parametrize` for tables of edge cases. Runs existing unittest tests too.
coverage.py / pytest-cov	7.x · Apache 2.0	Coverage measurement	Reports which lines and branches tests exercise; `--cov-fail-under=85` turns the target into a hard gate.
Git	2.x · GPL-2.0	Version control	Industry standard; full offline history; cheap branching for one-feature-per-branch work.
GitHub	cloud service	Hosting and collaboration	Public repository, issues, pull requests, reviews and Markdown rendering; required by the open-source track.
GitHub Actions	cloud service	Continuous integration	Runs tests on every push and pull request, free for public repositories, configured by one YAML file in the repo.
pyproject.toml	PEP 621	Packaging	The modern standard for package metadata; one line under `[project.scripts]` creates the `device-validator` command when the package is installed.
Markdown	—	Documentation	Rendered automatically by GitHub; used for README, rule catalog and contribution guide.

2.3 Development environment

Development happens in an isolated **virtual environment** so the project's packages never clash with other Python projects on the same machine.

```
setting up a development environment
# 1. get the code
$ git clone https://github.com/<user>/device-plugin-config-validator.git
$ cd device-plugin-config-validator

# 2. create and activate a virtual environment
$ python3 -m venv .venv
$ source .venv/bin/activate          # Windows: .venv\Scripts\activate

# 3. install the package in editable mode, with test tools
$ pip install -e ".[dev]"

# 4. run the tests with coverage
$ pytest --cov=device_validator --cov-report=term-missing
```

> TERM **Editable install** (`pip install -e`) links the installed package to your source folder, so code changes take effect immediately without reinstalling.

2.4 Why there is no database

The tool reads files, checks them, prints a report and exits. Nothing has to be remembered between runs, and the rule catalog is small enough to live in Python code (or, as a stretch goal, in a YAML file kept in the repository). Adding a database would bring installation steps, a schema to maintain and a new failure point, with no benefit to the user.

If the project later needs…	Recommended choice	Reason
A history of validation runs	**SQLite**	Built into Python (`sqlite3`), a single file, no server to run.
Shared results across a team	CI artefacts (JSON reports)	GitHub Actions can store each run's JSON report; no database needed.
Large, shared rule libraries	Versioned YAML rule files	Reviewable in pull requests like any other code.

2.5 Alternatives compared

Programming language

Option	Advantages	Limitations	Decision
Python	Fast to write and read; excellent for nested data; large testing ecosystem; matches the PRD.	Slower than compiled languages; needs a Python runtime.	Chosen
Go	Kubernetes and most device plugins are written in Go; single static binary; very fast.	Steeper learning curve for a 4-week, first-year project; more code for the same features.	Future option
TypeScript / Node.js	Large ecosystem; strong JSON support.	Uncommon in infrastructure tooling; needs Node on every machine.	Rejected
Rust	Very fast and memory-safe; single binary.	Hardest learning curve; slow to iterate for a short project.	Rejected

Libraries and approaches

Area	Alternative	Advantages	Limitations (why not chosen)
CLI	Click	Decorator-based, composable commands; polished help.	Third-party dependency for features argparse already covers.
	Typer	Builds a CLI from type hints with very little code.	Depends on Click; hides mechanics that are valuable to learn.
YAML	ruamel.yaml	YAML 1.2; preserves comments; line numbers for every node.	Larger, more complex API; slower; line numbers not required in v1 (listed as a future enhancement).
	StrictYAML	Refuses implicit typing (`yes` stays a string).	Rejects valid YAML features real configs use; smaller community.

Area	Alternative	Advantages	Limitations (why not chosen)
Validation	JSON Schema (`jsonschema`)	Standard, portable schema language; good tooling.	Generic messages; awkward to attach rule IDs, per-rule severity and fix hints; no published schemas for these plugins.
	Pydantic	Typed models with detailed errors.	Requires modelling each vendor's whole schema as classes; heavier dependency; less control over severity.
Testing	unittest only	Built in; no install.	More boilerplate; no built-in parametrisation; less readable failures.
CI	GitLab CI, Jenkins	Mature, flexible pipelines.	Code lives on GitHub; Actions needs no extra account or server.

Scoring the main decision points

Option	Ease of learning	Extra dependencies	Fit for custom rules & hints	Speed	Overall
Python + custom rule engine	●●●●●	●●●●○	●●●●●	●●●○○	Best fit
Python + JSON Schema	●●●●○	●●●○○	●●○○○	●●●○○	Good
Python + Pydantic	●●●○○	●●○○○	●●●○○	●●●●○	Good
Go + custom engine	●●○○○	●●●●●	●●●●●	●●●●●	Later

More filled dots = better. "Extra dependencies": more dots means fewer dependencies.

> SELECTION PRINCIPLE Prefer the Python standard library, and add a third-party package only when it does something the standard library cannot. The result is one runtime dependency (PyYAML), a fast install and code that is easy to audit.

03 Basic Project Architecture

How the code is organised into folders and modules, what each part is responsible for, and how the parts work together.

3.1 Architectural style

The tool follows a **pipes-and-filters** architecture inside a thin **layered** shell. Data flows through a fixed sequence of independent stages (the *filters*). Each stage receives the previous stage's output, transforms it and passes it on. A single orchestrator, the CLI, connects the stages and is the only part that talks to the outside world.

A. Pipeline
load → traverse → check → redact → report. Each stage is a plain function with clear input and output types.

B. Thin shell
The CLI reads arguments, calls the stages, prints and exits. Everything else is pure and testable.

C. Rules as data
Rule catalogs (profiles) describe *what* to check; the engine decides *how*. New rules need no new logic.

> TERM **Pure function**: a function whose result depends only on its inputs and which changes nothing outside itself: no printing, no files, no global variables. Pure functions are the easiest code to test and reason about.

3.2 Design principles

Principle	What it means here	Benefit
Single responsibility	The loader only loads, the engine only checks, the reporter only formats.	Each file is small and understandable on its own.
Pure stages, one noisy edge	Only `cli.py` prints, reads `sys.argv` or sets the exit code.	Every stage can be unit-tested directly.
Errors are data	Problems become `Finding` objects; exceptions are caught at module boundaries.	The tool never crashes on bad input.
Rules are data	Profiles list rules built from reusable check factories.	Adding a rule is one line.
Secure by default	Safe parsers only; values masked when a finding is created.	No code execution; no secret leaks.

Principle	What it means here	Benefit
Deterministic	Findings sorted by document position, then rule ID.	Stable tests and CI results.
One-way dependencies	Modules import only "downwards"; shared types live at the bottom.	No circular imports; easy refactoring.

3.3 Folder and file structure

```
device-plugin-config-validator/
├── .github/
│   ├── workflows/
│   │   └── ci.yml                # GitHub Actions: tests + coverage on every push / PR
│   ├── ISSUE_TEMPLATE/           # bug report and feature request forms
│   └── pull_request_template.md  # checklist shown on every new PR
├── src/
│   └── device_validator/         # the installable Python package
│       ├── __init__.py           # package marker, __version__
│       ├── __main__.py           # enables "python -m device_validator"
│       ├── cli.py                # argument parsing, orchestration, exit codes
│       ├── models.py             # Severity, Rule, Finding, Report
│       ├── paths.py              # path tuples, display strings, wildcard matching
│       ├── loader.py             # file checks + safe JSON / YAML parsing
│       ├── traverser.py          # depth-first walk → ordered node map
│       ├── engine.py             # applies rules → findings
│       ├── redact.py             # decides what may be shown for a value
│       ├── rules/
│       │   ├── __init__.py       # get_profile(name) → list[Rule]
│       │   ├── checks.py         # check factories: is_type, in_range, one_of
│       │   └── profiles/
│       │       └── intel_gpu.py  # first vendor rule catalog (mentor-approved)
│       └── reporters/
│           ├── __init__.py
│           ├── text.py           # human-readable terminal report
│           └── json.py           # machine-readable report (stretch)
├── tests/
│   ├── fixtures/
│   │   ├── valid/                # must pass with zero findings
│   │   ├── invalid/              # parse fine, break specific rules
│   │   └── malformed/            # broken syntax, empty, scalar, huge, too deep
│   ├── golden/                   # approved expected report outputs
│   ├── conftest.py               # shared pytest fixtures
│   └── test_*.py                 # one test module per source module
├── docs/
│   ├── RULES.md                  # full rule catalog with IDs and fixes
│   └── ARCHITECTURE.md           # this design, in Markdown
├── .gitignore                    # files Git must never track
├── CONTRIBUTING.md               # how to set up, branch, test and open a PR
├── LICENSE                       # open-source licence (e.g. MIT / Apache-2.0)
├── pyproject.toml                # metadata, dependencies, CLI entry point
└── README.md                     # front page: what, install, usage, rules, limits
```

3.4 Responsibilities of each folder and file

Folder / file	Responsibility	Depends on
`cli.py`	Defines commands and options; runs the pipeline for each file; chooses the reporter; computes the exit code. The only module allowed to print or exit.	all modules
`models.py`	Shared data types: `Severity`, `Rule`, `Finding`, `Report`.	nothing
`paths.py`	Represents field paths as tuples, formats them for display, parses rule patterns and matches wildcards.	nothing
`loader.py`	Checks existence, size and encoding; parses safely; validates the top-level type. Returns data or one fatal finding.	models

Folder / file	Responsibility	Depends on
`traverser.py`	Visits every node depth-first and builds an ordered map from path to value. Enforces depth and node limits.	paths, models
`engine.py`	For each rule, finds matching nodes and runs its check; detects missing keys; builds sorted findings.	paths, redact, models
`redact.py`	Masks sensitive keys and secret-looking values; summarises containers; shortens long strings.	paths
`rules/checks.py`	Reusable check factories that return small check functions.	nothing
`rules/profiles/`	One rule catalog per config family (e.g. Intel GPU plugin, NVIDIA plugin, SR-IOV).	checks, models, paths
`reporters/`	Turn `Report` objects into text or JSON strings. Never decide outcomes.	models
`tests/`	Unit, integration, golden-file, end-to-end, security and performance tests, plus fixtures.	the package
`.github/`	CI workflow and contribution templates for issues and pull requests.	—
`docs/`	Rule catalog and architecture notes, kept next to the code they describe.	—
`pyproject.toml`	Package name, version, Python requirement, dependencies and the `device-validator` command.	—
`README.md`	The repository's front page (see Section 5.2).	—
`.gitignore`	Keeps generated, local and private files out of Git (see Section 5.2).	—

> TERM `src/` **layout**: placing the package inside `src/` means tests must import the *installed* package, not a folder that happens to be nearby. Packaging mistakes (a missing file, a wrong entry point) are caught immediately.

Key configuration file: `pyproject.toml`

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "device-plugin-config-validator"
version = "0.1.0"
description = "Offline validator for Kubernetes device plugin configuration files"
requires-python = ">=3.10"
dependencies = ["PyYAML>=6.0"]

[project.optional-dependencies]
dev = ["pytest>=8", "pytest-cov>=5"]

[project.scripts]
device-validator = "device_validator.cli:main"   # creates the terminal command
```

3.5 Crucial components and their contracts

Each component has a small, explicit **contract**: the function other modules call, what it accepts and what it promises to return.

Component	Main function (signature)	Promise
CLI	`main(argv: list[str] | None = None) -> int`	Returns the exit code instead of calling `sys.exit`, so tests can call it directly.
Loader	`load(path: Path, max_bytes: int) -> LoadResult`	Never raises. Returns either `data` (a dict) or exactly one fatal `Finding`.
Traverser	`walk(data: dict, max_depth=64, max_nodes=100_000) -> NodeMap`	Every node appears once, in document order. Raises a typed `LimitExceeded` that the CLI converts to a fatal finding.
Rule profile	`get_profile(name: str) -> list[Rule]`	Returns a fixed, ordered rule list; unknown names produce a clear usage error.

Component	Main function (signature)	Promise
Engine	`evaluate(nodes, rules, file) -> tuple[list[Finding], int]`	Sorted findings plus the number of rule evaluations performed. Values are already redacted.
Redactor	`summarize(path: Path, value: Any) -> str`	Always returns a short, safe string; never returns raw secrets or container contents.
Reporter	`render(reports: list[Report], color: bool) -> str`	Pure formatting: same reports always give the same text.

3.6 How the components interact

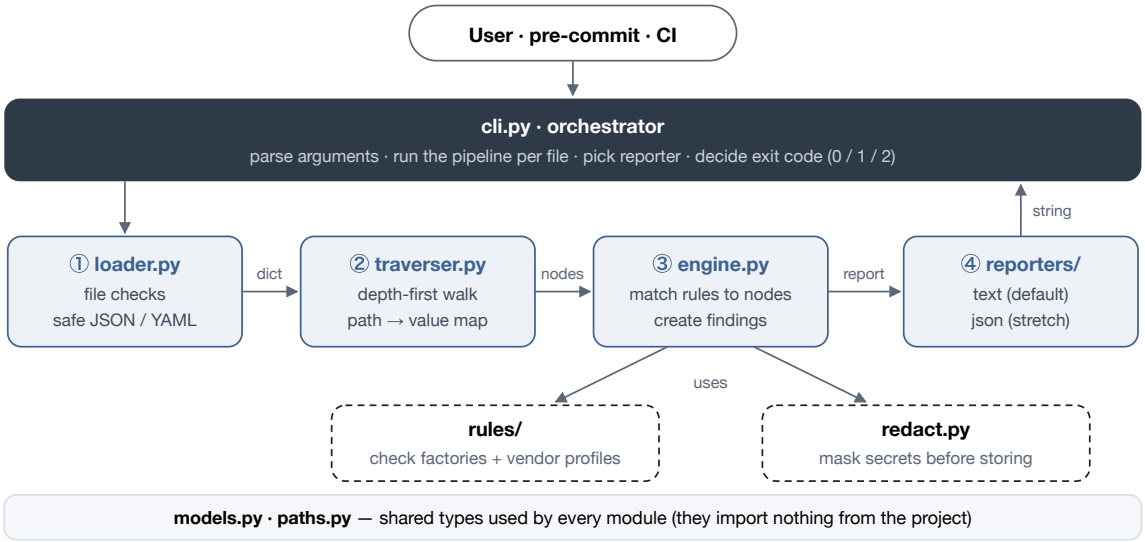


Figure 4. Component architecture. Numbers show the order of the pipeline; dashed boxes are helpers used by the engine.

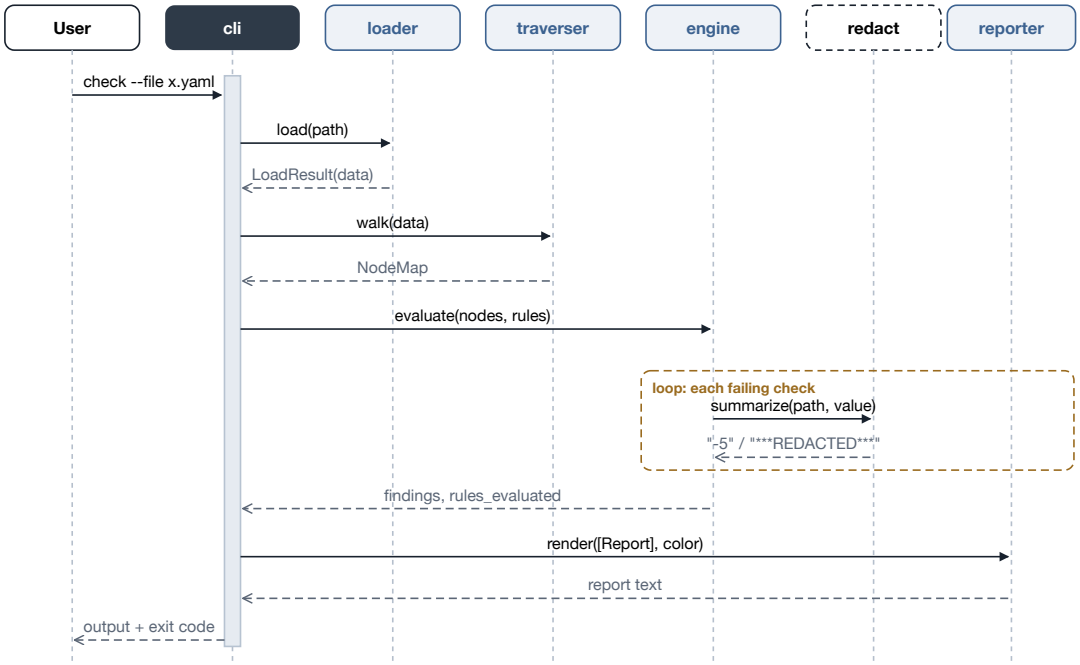


Figure 5. Sequence diagram for one file. Solid arrows are calls; dashed arrows are returned values. Only the CLI talks to the user.

3.7 Data model

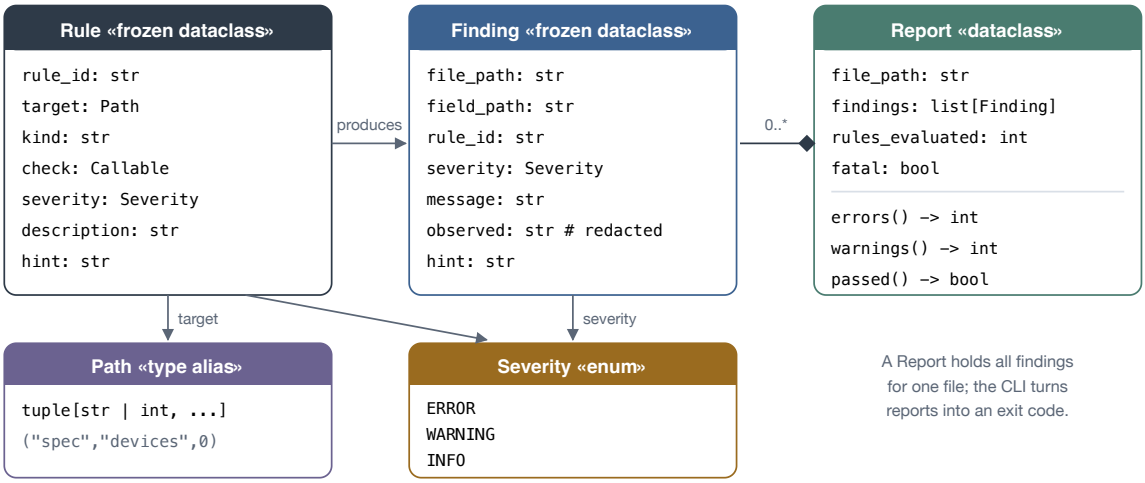


Figure 6. Class diagram of the core data model. A rule *produces* findings when its check fails; a report *aggregates* the findings for one file.

Field	Meaning	Example
`Rule.target`	Pattern of paths the rule applies to; `*` matches any key or list index.	`spec.devices[*].memory.limit`
`Rule.kind`	Which family of check: `required`, `type`, `range` (optionally `one_of`).	`range`
`Rule.check`	Function returning a problem message, or `None` if the value is fine.	`in_range(1, 64)`
`Finding.message`	What is wrong, in plain language.	`must be between 1 and 64`
`Finding.observed`	A safe, shortened view of the actual value.	`-5`, `***REDACTED***`
`Finding.hint`	How to fix it.	`Set a value between 1 and 64`
`Report.fatal`	True when the file could not be loaded at all.	`True` for a syntax error

3.8 The path model

Every value in a document has an **address**. Internally, an address is a *tuple* of keys and list indexes; it is turned into a readable string only for display.

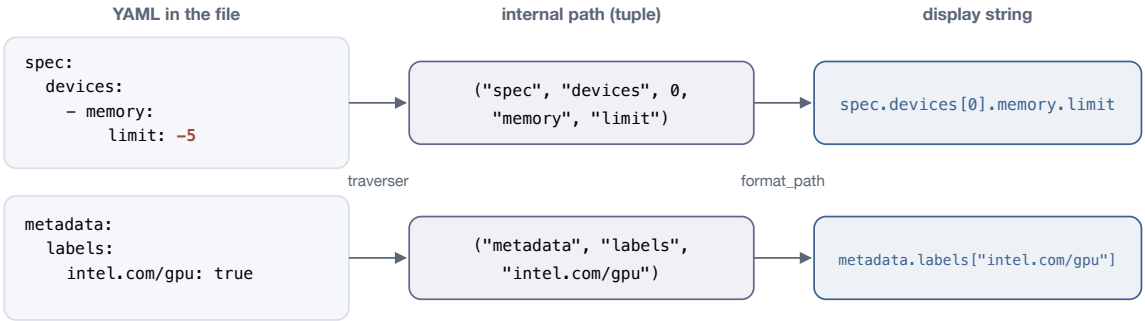


Figure 7. Why tuples: the key `intel.com/gpu` contains dots. As a plain string, `labels.intel.com/gpu` would be ambiguous; as a tuple it is exact, and the display form quotes it.

Rule pattern	Concrete path	Match?
`spec.devices[*].memory.limit`	`spec.devices[0].memory.limit`	yes index 0
`spec.devices[*].memory.limit`	`spec.devices[3].memory.limit`	yes any index
`spec.devices[*].memory.limit`	`spec.devices[0].memory`	no too short
`sharing.*.resources[*].replicas`	`sharing.timeSlicing.resources[1].replicas`	yes key wildcard

3.9 Rule catalog design

Rules are built from small **check factories**: functions that create and return a ready-to-use check. For example, `in_range(1, 64)` returns a function that already knows the limits 1 and 64.

Kind	Factory	Fails when…	Example message
required	(handled by the engine)	A parent exists but the child key is missing.	`required key is missing`
type	`is_type(int)`	The value has the wrong type. **Booleans are rejected where numbers are expected**, because in Python `isinstance(True, int)` is `True`.	`expected integer, got boolean`
range	`in_range(1, 64)`	A number is outside the bounds. Non-numbers are skipped, so a wrong type is never reported twice.	`must be between 1 and 64`
one_of (optional)	`one_of("none", "single", "mixed")`	The value is not in an allowed set.	`must be one of: none, single, mixed`

Rule ID scheme

Prefix	Meaning	Default severity	Exit code
`F0xx`	Fatal: file or parse problem; validation cannot continue	ERROR	2
`K0xx`	Required key missing	ERROR	1
`T0xx`	Wrong data type	ERROR	1
`R0xx`	Number out of range	ERROR or WARNING	1 / 0*
`U0xx`	Unknown key (vendor extension), shown with `--verbose`	INFO	0

* Warnings exit with 0 unless `--strict` is used. Unknown keys are never errors: vendors add custom fields, and flagging them would create false positives (PRD risk 5.1).

Example catalog entries (illustrative, pending mentor approval)

ID	Target	Check	Severity	Hint
`K001`	`spec`	required	ERROR	Add a top-level `spec` section
`K002`	`spec.devices[*].name`	required	ERROR	Give every device a unique name
`T001`	`spec.devices[*].name`	is_type(str)	ERROR	Use text, e.g. `"gpu-0"`
`T004`	`spec.resourceManager`	is_type(bool)	ERROR	Use `true` or `false`
`R003`	`spec.devices[*].memory.limit`	in_range(1, 64)	ERROR	Set a value between 1 and 64
`R007`	`spec.logLevel`	in_range(0, 4)	WARNING	Use 0–4 unless debugging

3.10 Error-handling design

Fatal problems stop validation of *that file only*; other files in the same run are still checked. Each fatal problem has its own rule ID, so users and tests can tell them apart.

ID	Condition	Detected by	Message shown to the user
`F001`	File missing or not a regular file	`Path.is_file()`	file not found: gpu.yaml
`F002`	File larger than the limit (1 MiB)	`stat().st_size`	file is 4.2 MiB; limit is 1 MiB (--max-size)
`F003`	Unreadable: permissions or not UTF-8	`PermissionError`, `UnicodeDecodeError`	file could not be read as UTF-8 text
`F004`	Empty or whitespace-only document	`text.strip() == ""`	document is empty
`F005`	Syntax error, or several YAML documents	`JSONDecodeError`, `yaml.YAMLError`	syntax error at line 12, column 5
`F006`	Top level is not a mapping	`isinstance(data, dict)`	top level must be a mapping, got a list

ID	Condition	Detected by	Message shown to the user
`F007`	Nesting deeper than 64 or more than 100,000 nodes	traverser limits	document exceeds the nesting limit (64)

> **IMPORTANT** PyYAML's error messages include a snippet of the broken line, and that line could contain a password. The loader therefore never copies parser text into a finding: it keeps only the line and column numbers and writes its own message.

3.11 Security design

Threat	What could happen	Mitigation in the design
Code execution via YAML	Unsafe loaders can build arbitrary Python objects from tags like `!!python/object`.	Only `SafeLoader` / `CSafeLoader` are ever used; a test asserts that unsafe tags are rejected.
"Billion laughs" aliases	Nested YAML aliases make a tiny file expand into billions of nodes during traversal.	Traverser stops at 100,000 nodes with `F007`.
Deep nesting	Thousands of nested levels exhaust Python's recursion limit (~1,000) and crash.	Depth limit of 64 with `F007`; JSON `RecursionError` is also caught.
Huge files	Very large inputs use excessive memory or time.	Size is checked with `stat()` *before* reading; default limit 1 MiB.
Secret leakage	Passwords or tokens printed to terminals and CI logs.	Redaction when a finding is created; containers are never printed; parser snippets are never copied.
Terminal injection	Control characters in values could manipulate the terminal.	Observed values are shown with `repr()`, which escapes control characters.

Redaction rules

Situation	Shown as
Key name contains a sensitive word part: `password`, `passwd`, `secret`, `token`, `key` (as in `apiKey`, `private_key`), `credential`, `cert`	`***REDACTED***`
Value looks like a secret: PEM block (`-----BEGIN`), long base64-like string (32+ characters), `Bearer …`	`***REDACTED***`
Dictionary or list	`mapping with 3 keys` / `list of 2 items`
Long string	first 60 characters + `…`
Anything else	`repr(value)`, e.g. `-5`, `'yes'`, `True`

Key names are split into word parts (`apiKey` → `api`, `key`), so innocent keys such as `keyboard` or `monkey` are not masked.

3.12 Performance design

Stage	Budget (500-line file)	How it is achieved
Python start-up + imports	~40–60 ms	Few imports; no heavy dependencies.
Parsing	~10–40 ms	`CSafeLoader` (C) when available; JSON parser is already in C.
Traversal	< 5 ms	Each node visited exactly once.
Rule evaluation	< 10 ms	~2,000 nodes × ~30 rules of cheap tuple comparisons.
Reporting	< 5 ms	Simple string building.
Total	**< 200 ms target**	Verified by a timed test; memory stays well under 30 MB.

04 Project Workflow

How a single command travels through the system: from user input, through each processing step, to the report and exit code.

4.1 End-to-end flowchart

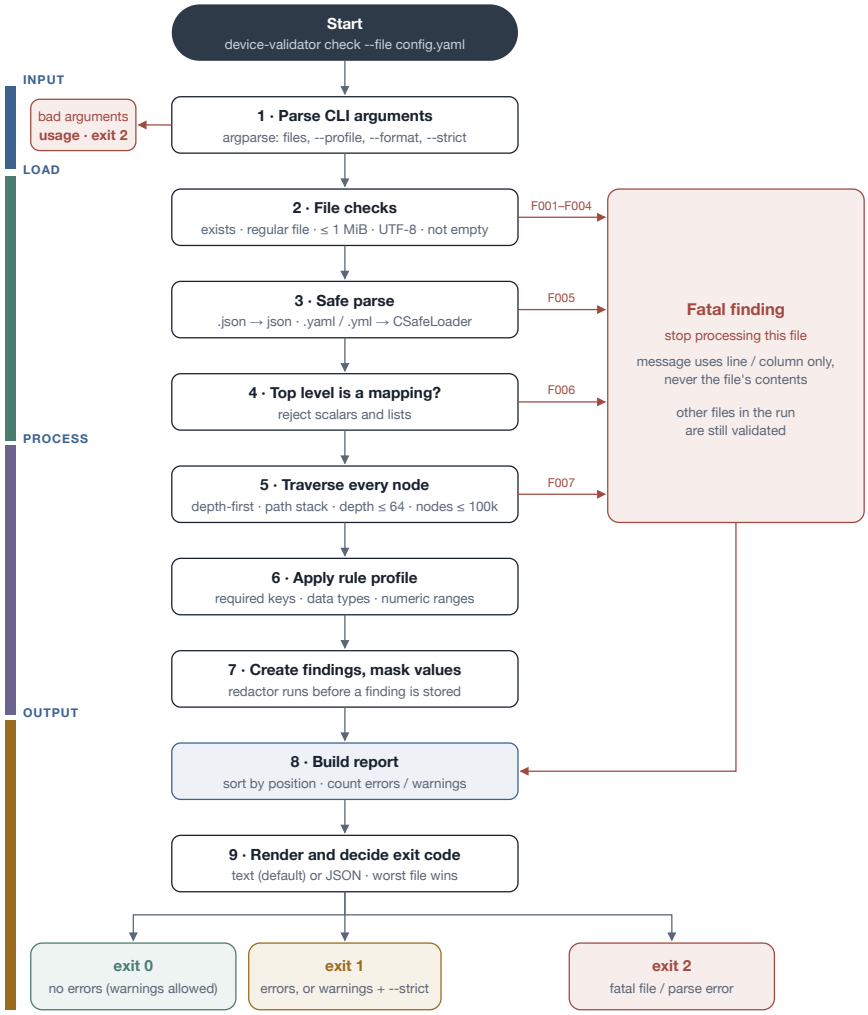


Figure 8. Application workflow from user input to report and exit code. Red paths are fatal problems that stop validation of the current file.

4.2 Inputs

```
command syntax
$ device-validator check --file PATH [--file PATH ...] [--profile NAME]
                         [--format text|json] [--strict] [--verbose] [--max-size BYTES]
$ device-validator rules [--profile NAME]      # list the rule catalog
$ device-validator --help
```

Input / option	Default	Meaning
`--file PATH`	required	A JSON (`.json`) or YAML (`.yaml` , `.yml`) file. Can be repeated; a directory is allowed as a stretch goal.
`--profile NAME`	first profile	Which rule catalog to apply (e.g. `intel-gpu`).
`--format`	`text`	`text` for people, `json` for machines (stretch).
`--strict`	off	Treat warnings as failures (exit 1) (stretch).
`--verbose`	off	Also show INFO findings such as unknown vendor keys.
`--max-size`	1 MiB	Largest file the loader will accept.
Environment: `NO_COLOR`	unset	When set, the report never uses colour.

Input / option	Default	Meaning
Rule catalog	built in	Mentor-approved rules defining required keys, data types and numeric bounds.

4.3 Processing steps in detail

#	Step	What happens	Why
1	**Parse arguments**	`argparse` turns the command words into an `Options` object. Invalid input prints usage and exits 2.	Fail fast with a clear message.
2	**File checks**	Exists and is a regular file; size checked with `stat()` before reading; read as UTF-8; not empty.	Avoid reading huge or binary files.
3	**Safe parse**	Extension decides the parser; anything that is not `.json` uses the YAML parser (which also reads JSON).	Safe loaders never run code.
4	**Top-level check**	The parsed result must be a dictionary.	Rules address keys; a bare value or list has none.
5	**Traverse**	Depth-first walk; push a key when entering a node, pop it when leaving; record every node in an ordered map.	Gives each value an exact address; enables missing-key checks.
6	**Apply rules**	For each rule, find matching paths; run its check; for "required" rules, check each matching parent for the child key.	Generic engine; rules stay data.
7	**Mask values**	Each finding's observed value goes through `redact.summarize()` before the finding is created.	Secrets never stored, so never printed.
8	**Build report**	Sort findings by document position, then rule ID; count errors, warnings, rules evaluated.	Deterministic, readable output.
9	**Render & exit**	Reporter turns reports into text or JSON; CLI prints it and returns the exit code.	Scripts and CI can act on the result.

4.4 Data transformation walkthrough

The example below follows one small file through the pipeline. It shows what the data looks like after each stage.

Stage 0 · input file `gpu-config.yaml`

```
spec:
  logLevel: 9
  devices:
    - name: gpu-0
      memory:
        limit: -5
    - memory:
        limit: 16
  credentials:
    token: abc123secret
```

Stage 1 · after loading (Python dict)

```
{"spec": {
   "logLevel": 9,
   "devices": [
     {"name": "gpu-0", "memory": {"limit": -5}},
     {"memory": {"limit": 16}}],
   "credentials": {"token": "abc123secret"}}}
```

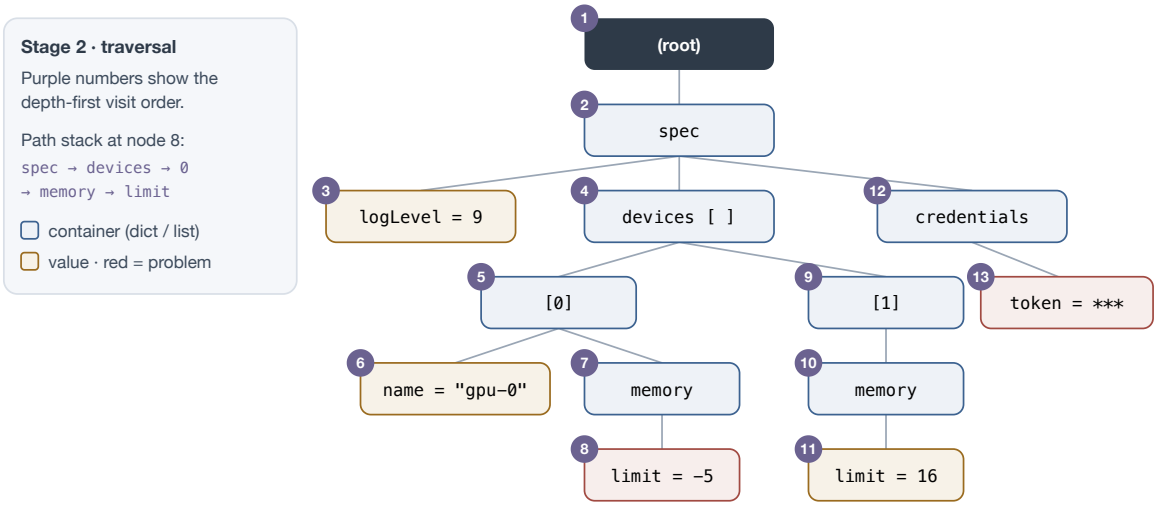


Figure 9. The document as a tree. The traverser visits nodes depth-first, keeping a stack of the keys it has passed through, so every value gets an exact address.

Stage 3 · node map (excerpt)

#	Path	Value
3	`spec.logLevel`	`9`
5	`spec.devices[0]`	mapping, 2 keys
6	`spec.devices[0].name`	`'gpu-0'`
8	`spec.devices[0].memory.limit`	`-5`
9	`spec.devices[1]`	mapping, 1 key
11	`spec.devices[1].memory.limit`	`16`
13	`spec.credentials.token`	`***REDACTED***`

Stage 4 · rule results

Rule	Applied to	Result
`R007`	node 3 (9)	WARNING
`K002`	nodes 5, 9	5 ok · 9 missing
`T001`	node 6	ok
`R003`	node 8 (−5)	ERROR
`R003`	node 11 (16)	ok

The `token` value is never printed. If a rule ever reported it, it would appear only as `***REDACTED***`.

4.5 Outputs

Human-readable report (default)

```
device-validator check --file gpu-config.yaml
$ device-validator check --file gpu-config.yaml

gpu-config.yaml
  WARNING  spec.logLevel                R007  above recommended maximum 4 (got 9)
           ↳ Use 0–4 unless you are debugging
  ERROR    spec.devices[0].memory.limit  R003  must be between 1 and 64 (got -5)
           ↳ Set a value between 1 and 64
  ERROR    spec.devices[1].name          K002  required key is missing
           ↳ Give every device a unique name
──────────────────────────────────────────────────────────────────────────────
  14 rules evaluated · 2 errors · 1 warning · FAILED

$ echo $?
1
```

```
a clean file
$ device-validator check --file intel-gpu-ok.yaml
intel-gpu-ok.yaml  ✓ PASSED  14 rules evaluated · no problems found
```

Machine-readable report (`--format=json` , stretch)

```
{
  "files": [{
    "file": "gpu-config.yaml", "fatal": false, "rules_evaluated": 14,
    "summary": {"errors": 2, "warnings": 1, "infos": 0},
    "findings": [
      {"rule_id": "R007", "severity": "warning", "path": "spec.logLevel",
       "message": "above recommended maximum 4", "observed": "9",
       "hint": "Use 0–4 unless you are debugging"},
      {"rule_id": "R003", "severity": "error", "path": "spec.devices[0].memory.limit",
       "message": "must be between 1 and 64", "observed": "-5",
       "hint": "Set a value between 1 and 64"},
      {"rule_id": "K002", "severity": "error", "path": "spec.devices[1].name",
       "message": "required key is missing", "observed": "",
       "hint": "Give every device a unique name"}
    ]}],
  "exit_code": 1
}
```

Exit codes

Code	Meaning	When it happens	What CI does
`0`	**Valid**	No errors in any file (warnings allowed unless `--strict`).	Check passes
`1`	**Validation errors**	At least one ERROR finding, or a WARNING with `--strict` .	Check fails; merge blocked
`2`	**Fatal**	A file could not be read or parsed, or the command was used incorrectly.	Check fails; fix input first

With several files, the worst result decides the exit code (2 beats 1 beats 0).

4.6 Scenario matrix

These scenarios become end-to-end tests; each fixture file in `tests/fixtures/` maps to one row.

Input scenario	Expected findings	Exit
Fully valid config	None; "PASSED"	0
Out-of-range value marked as warning	One `R…` WARNING	0
Same file with `--strict`	One `R…` WARNING	1
Missing required key	`K…` ERROR at the parent's path	1
`replicas: true` where an integer is required	`T…` ERROR "expected integer, got boolean"	1
Wrong type *and* range rule on the same field	Only the `T…` finding (no duplicate range error)	1
Bad YAML indentation	`F005` with line and column	2
Empty file / only comments	`F004`	2
File containing just `42` or a list	`F006`	2
Path that does not exist	`F001`	2
"Billion laughs" alias bomb	`F007` , no hang	2
Secrets in a failing field	Finding shows `***REDACTED***` ; secret absent from output	1
Two files: one clean, one fatal	Clean report + fatal report	2

05 Git and GitHub Documentation

The version-control concepts and collaboration workflow used to build, review and publish this project.

5.1 Git and GitHub, and how they differ

Git: the tool

Git is a free, open-source *distributed version control system*, created by Linus Torvalds in 2005 to manage Linux kernel development. It runs on your own computer and records the full history of a project's files as a series of snapshots. Every copy of a Git repository contains the entire history, so you can commit, branch, compare and restore work completely offline.

GitHub: the platform

GitHub is a cloud platform (launched in 2008, owned by Microsoft since 2018) that hosts Git repositories online. On top of Git it adds a web interface and collaboration tools: issues, pull requests, code review, GitHub Actions automation, project boards, wikis, Pages and Markdown rendering.

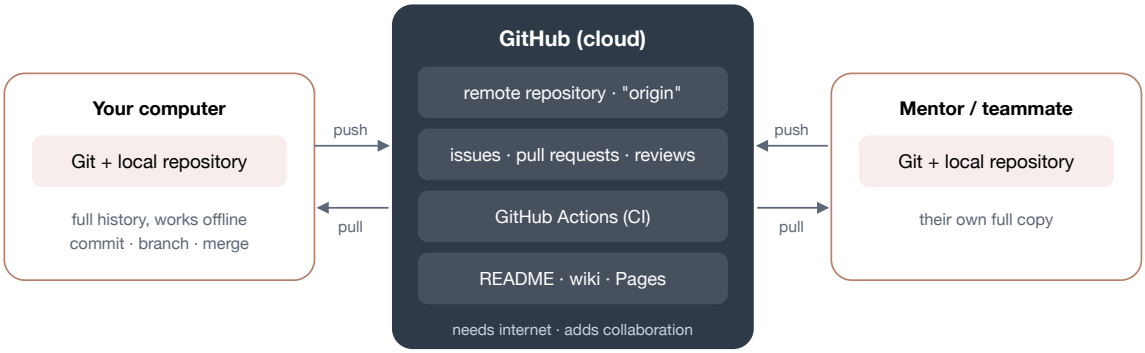


Figure 10. Git runs on every developer's machine; GitHub is the shared meeting point where copies are synchronised and collaboration happens.

Aspect	Git	GitHub
What it is	Software: a command-line version control system	Service: a website and platform for hosting repositories
Where it runs	Locally, on your computer	In the cloud, used through a browser, CLI (`gh`) or apps
Internet needed?	No (except to push / pull)	Yes
Main purpose	Track changes, branch, merge, restore history	Share code, collaborate, review, automate, document
Key features	commits, branches, merges, tags, diff, log	issues, pull requests, Actions, Projects, wiki, Pages, security alerts
Cost / ownership	Free, open source (GPL-2.0)	Free tier for public repos; owned by Microsoft
Alternatives	Mercurial, Subversion (SVN), Perforce	GitLab, Bitbucket, Gitea, Azure DevOps
Analogy	A camera that takes snapshots of your work	A shared online album where the team views, discusses and combines the snapshots

5.2 Core concepts

Repository

A **repository** ("repo") is a project folder whose complete history is tracked by Git. The history lives in a hidden `.git/` folder inside the project. A **local repository** is the copy on your computer; a **remote repository** is a copy hosted elsewhere, usually on GitHub, and is conventionally called `origin`. A repository can be *public* (anyone can read it, as required for this open-source project) or *private*.

Commit

A **commit** is a saved snapshot of the project at one moment. Each commit records the staged changes, an author, a timestamp, a message explaining the change, a pointer to its *parent* commit, and a unique ID (a SHA hash, usually shortened to 7 characters such as `a1b2c3d`). Because each commit points to its parent, commits form a chain: the project's history.

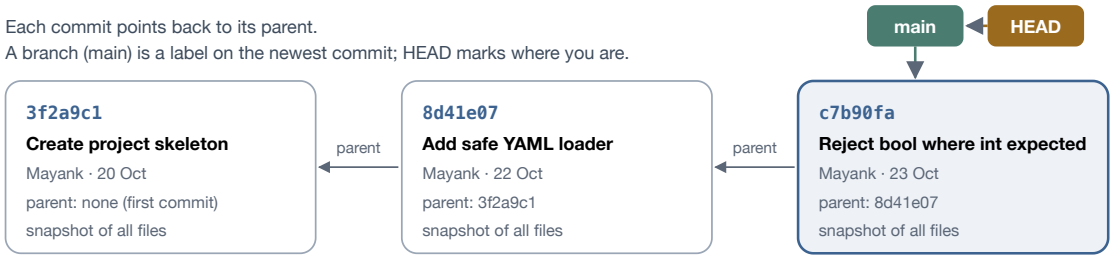


Figure 11. Anatomy of commits. Commit IDs, names and dates are illustrative.

Good commit messages	Poor commit messages
`Add safe YAML loader with size limit`	`update`
`Reject bool where int is expected (T00x)`	`fixed stuff`
`Mask apiKey and token values in findings`	`changes`

Convention used: short summary in the imperative mood ("Add", "Fix", "Remove"), ≤ 60 characters, optional body explaining *why*.

Branch

A **branch** is an independent line of development. Technically it is just a movable label that points at a commit, so creating one is instant and costs nothing. `main` holds the stable, working version of the project; each new feature or fix is developed on its own branch, for example `feature/redaction`, so unfinished work never breaks `main`.

Merge

A **merge** combines the commits of one branch into another, typically a finished feature branch into `main`. Git performs one of two kinds of merge:

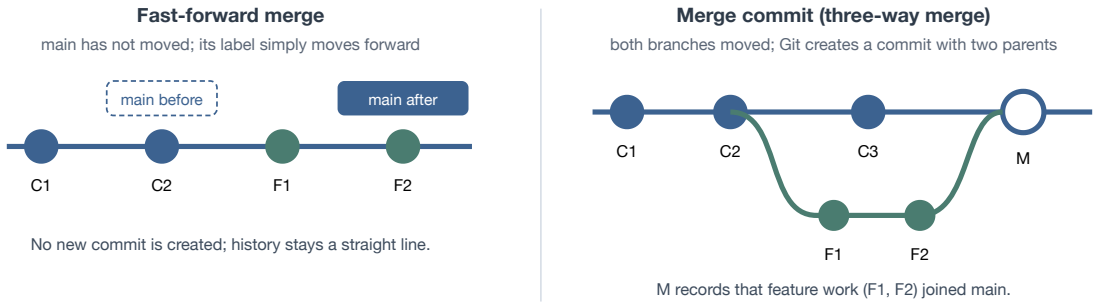


Figure 12. The two kinds of merge.

If both branches changed *the same lines* in different ways, Git cannot decide which version is right and reports a **merge conflict**. It marks the competing versions in the file; a person chooses the correct result, removes the markers and commits.

Conflict as Git shows it

```
<<<<<<< HEAD
MAX_DEPTH = 64
=======
MAX_DEPTH = 100
>>>>>>> feature/limits
```

After resolving

```
MAX_DEPTH = 64   # agreed with mentor
```

Then: `git add traverser.py` and `git commit` to complete the merge.

README file

`README.md` is the front page of a repository: GitHub renders it automatically below the file list. For many visitors it is the only file they read, so it must explain quickly what the project does and how to use it. This project's README will contain:

1. **What & why**
   One-paragraph description, the problem it solves, a status badge from CI.
2. **Install**
   Python requirement, `pip install` command, development setup.
3. **Usage**
   CLI reference with options, example commands and sample output.

4. Rules
Link to `docs/RULES.md` : every rule ID, severity and fix.

5. Examples
A clean config and an error-bearing config with their reports.

6. Limits & licence
Static checks only, not hardware certification; contributing guide; licence.

.gitignore

`.gitignore` is a text file listing files and folders Git must **never track**. It keeps the repository small and clean, and prevents accidental publication of private or secret files. Each line is a pattern:

This project's .gitignore (excerpt)

```
# private planning notes — never publish
private-notes/

# Python build and cache files
__pycache__/
*.py[cod]
build/
dist/
*.egg-info/

# virtual environments
.venv/

# test and coverage output
.pytest_cache/
.coverage
htmlcov/

# operating-system files
.DS_Store
```

Pattern rules

Pattern	Meaning
`name/`	Ignore a folder and everything inside it
`*.pyc`	`*` matches any characters
`*.py[cod]`	`[...]` matches one of the listed characters
`# text`	Comment
`!keep.txt`	Exception: do track this file

> **NOTE** `.gitignore` only affects *untracked* files. A file already committed must be removed with `git rm --cached` before it is ignored.

5.3 Everyday operations

Git moves changes through four areas. Once these are clear, every everyday command makes sense.

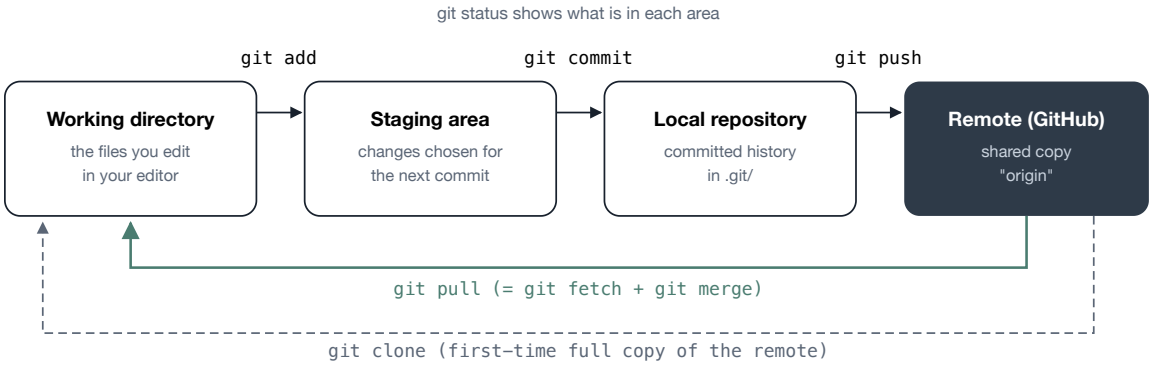


Figure 13. The four areas of Git and the commands that move changes between them.

Operation	Command	Purpose and when to use it
Clone	`git clone <url>`	Download a complete copy of a remote repository, including its whole history, and link it to `origin` . Done once per machine.
Status	`git status`	Show which files are modified, staged or untracked. Run it before every commit.
Stage	`git add <file>` · `git add -p`	Choose exactly which changes go into the next commit; `-p` lets you pick individual parts of a file. Staging keeps commits small and focused.
Commit	`git commit -m "Add range check"`	Save the staged changes as a snapshot in the local history.
Push	`git push -u origin <branch>`	Upload local commits to GitHub so others can see them and CI can test them. `-u` remembers the link for later pushes.

Operation	Command	Purpose and when to use it
Pull	`git pull`	Download new commits from GitHub and merge them into the current branch. Do it before starting new work.
Fetch	`git fetch`	Download new commits without changing your files, so you can inspect them first.
Branch	`git switch -c feature/x`	Create a new branch and move onto it.
Merge	`git merge feature/x`	Bring another branch's commits into the current branch.
History	`git log --oneline --graph`	Show past commits and how branches joined.
Compare	`git diff` · `git diff --staged`	Show unstaged or staged changes line by line.

A typical working session

```
building one feature from start to finish
$ git switch main && git pull              # start from the latest main
$ git switch -c feature/redaction          # new branch for this feature
  … edit src/device_validator/redact.py and tests/test_redact.py …
$ pytest                                    # make sure everything passes
$ git status                                # review what changed
$ git add src/device_validator/redact.py tests/test_redact.py
$ git commit -m "Mask sensitive keys in finding values"
$ git push -u origin feature/redaction      # upload the branch
  … open a pull request on GitHub, wait for CI and review, then merge …
```

5.4 Issues and pull requests

Issues

An **issue** is a tracked item in a repository's to-do list and discussion board. Each issue describes one bug, task or idea and can carry *labels* (`bug` , `enhancement` , `documentation` , `good first issue`), an *assignee*, a *milestone* and comments. Every issue gets a number, such as `#12` , that can be referenced anywhere on GitHub.

Open **Implement secret redaction in findings** #12

enhancement security Milestone: Week 3 · Assignee: Mayank Singhal

Goal: no finding may contain a secret value. **Acceptance criteria:** keys such as `password` , `token` , `apiKey` are masked; PEM blocks and long base64 strings are masked; containers are summarised; a test searches all output for planted secrets.

Pull requests

A **pull request (PR)** is a proposal to merge one branch into another, usually a feature branch into `main` . It shows the exact changes (the *diff*), lets reviewers comment on individual lines and suggest edits, runs automated CI checks, and keeps a permanent record of the discussion. Writing `Closes #12` in the description links the PR to its issue and closes the issue automatically when the PR is merged.

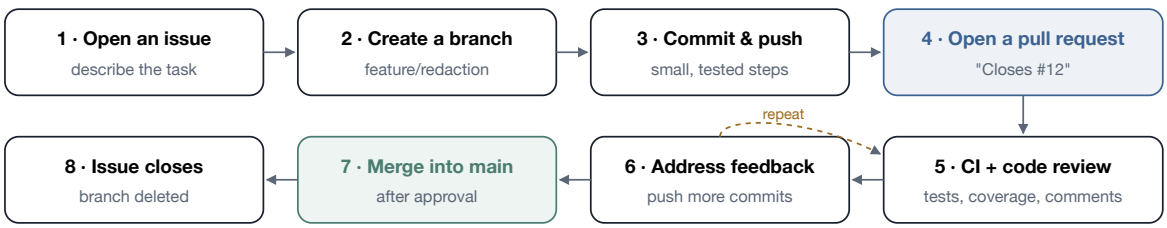


Figure 14. The life of a change: from issue to merged pull request.

Pull request template used in this project (`.github/pull_request_template.md`)

```
## What does this PR do?
Short description of the change.  Closes #<issue number>

## How was it tested?
- [ ] Unit tests added or updated
- [ ] `pytest` passes locally
- [ ] Coverage is still ≥ 85%

## Checklist
- [ ] No secrets or private files committed
- [ ] docs/RULES.md updated if rules changed
- [ ] README updated if CLI behaviour changed
```

5.5 How GitHub supports collaboration, code management and documentation

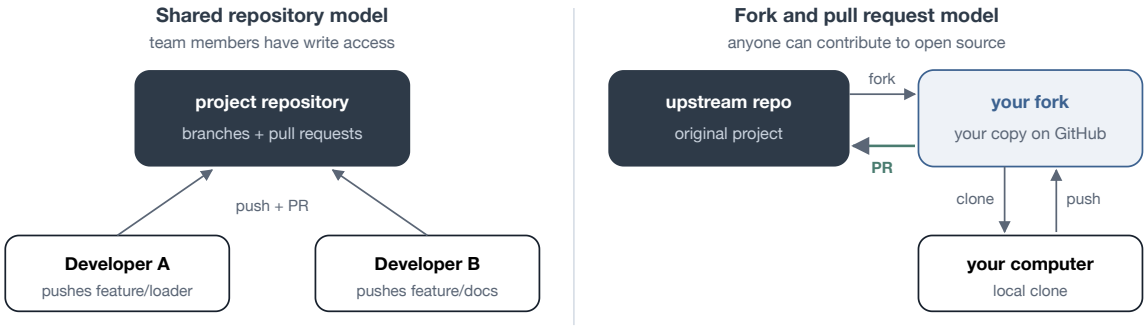


Figure 15. Two collaboration models. This OJT uses the shared model; outside contributors to the public repository would use forks.

Area	GitHub features	Use in this project
Collaboration	Pull request reviews with line comments and suggested changes; issues and discussions; @mentions and notifications; forks.	Mentor reviews each feature PR; questions tracked as issues.
Code management	Full history and *blame* view; branch protection rules; tags and releases; compare views.	`main` protected: merge only via PR with green CI.
Automation	GitHub Actions workflows; status checks on PRs; Dependabot dependency alerts.	Tests + coverage on Python 3.10–3.13 for every push.
Project management	Labels, milestones, Projects boards (To do → In progress → Done).	One milestone per week, matching mentor checkpoints.
Documentation	Automatic Markdown rendering, wikis, GitHub Pages, issue and PR templates.	README, RULES.md, CONTRIBUTING.md, PR template.
Open-source community	Licence detection, `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`, stars, forks.	Licence and contribution guide from day one.
Security	Secret scanning, Dependabot alerts, private vulnerability reporting.	A second safety net behind `.gitignore`.

Continuous integration with GitHub Actions



Figure 16. The CI pipeline that runs on every push and pull request.

.github/workflows/ci.yml

```yaml
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12", "3.13"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
      - run: pip install -e ".[dev]"
      - run: pytest --cov=device_validator --cov-fail-under=85
```

5.6 How this project uses Git and GitHub

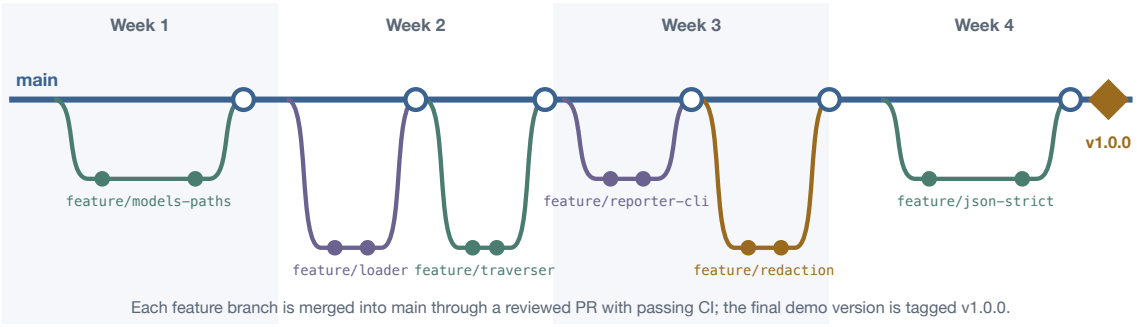


Figure 17. Planned branching strategy (GitHub flow) across the four weeks. White circles are merge points on `main`.

Convention	Rule	Example
Default branch	`main` is always working and protected; no direct pushes once CI exists.	—
Branch names	`feature/…`, `fix/…`, `docs/…`, `test/…`; short and lowercase.	`fix/bool-accepted-as-int`
Commits	Small, focused, imperative mood, tests passing.	`Add F006 for scalar top level`
Issues	One issue per PRD feature; labelled; assigned to a weekly milestone.	`#7 Build depth-first traverser`
Pull requests	One feature per PR; template filled in; links its issue; CI green; mentor approval.	`Closes #7`
Releases	Tag the version demonstrated at final evaluation.	`v1.0.0`
Private files	Planning notes live in an ignored folder and are never committed.	`private-notes/` in `.gitignore`

> **HABIT** Run `git status` before every `git add` and every `git commit`. It is the simplest way to make sure only the intended files, and no private or secret files, are about to be published.

06 Technical Justification and Conclusion

Why the architecture and technologies were chosen, how they support maintainability, scalability and growth, and how the project will be delivered.

6.1 Decision record

Each significant decision is recorded with the alternatives that were considered and its consequences, so future contributors understand *why* the project looks the way it does.

#	Decision	Alternatives considered	Rationale	Consequence
D1	**Python 3.10+**	Go, TypeScript, Rust	Fast development, readable code, natural fit for nested data, specified by the PRD.	Needs a Python runtime; Go port possible later.
D2	**Standard library first**	Click, Typer, Pydantic	Fewer dependencies, easier install and audit, more learning value.	Only PyYAML is required at runtime.
D3	**Safe loaders only**	`yaml.load`, ruamel.yaml	Input files are untrusted; safe loaders cannot execute code.	No custom YAML tags supported.
D4	**Custom rule engine**	JSON Schema, Pydantic	Full control of rule IDs, severities, messages and hints; no published schemas exist.	Engine code to write and test.
D5	**Pipeline of pure stages**	One large function, class hierarchy	Each stage is testable alone; only the CLI has side effects.	Slightly more files.
D6	**Paths as tuples**	Dot-separated strings	Kubernetes keys contain dots; tuples are unambiguous.	Display formatting needed.
D7	**Vendor rule profiles**	One global rule list	Different vendors use different config shapes; avoids false positives.	Profile chosen with `--profile`.
D8	**Redact at finding creation**	Redact in the reporter	Secrets are never stored, so no reporter can leak them.	Every new output format is safe by default.
D9	**Hard input limits**	Trust input size	Guarantees the "zero crashes" goal against hostile files.	Very large configs need `--max-size`.
D10	**No database**	SQLite, PostgreSQL	Nothing persists between runs.	Simple install; history could be added later.
D11	**GitHub + Actions**	GitLab CI, Jenkins	Code, reviews and CI on one platform at no cost.	Workflow lives in the repository.

6.2 How the architecture supports quality attributes

Quality	Architectural mechanism	Evidence
Maintainability	Single-responsibility modules; one-way dependencies; typed dataclasses; rules kept as data.	Small files; no circular imports; ≥ 85% coverage.
Testability	Pure stages; `main()` returns an exit code; fixtures for every scenario.	Unit, golden-file and end-to-end test suites.
Reliability	Errors converted to findings at module boundaries; input limits.	Malformed-input fixtures; zero unhandled exceptions.
Security	Safe loaders; early redaction; no parser snippets; `repr()` output.	Planted-secret test; unsafe-tag test.
Performance	Single traversal; cheap matching; C-accelerated parsing.	Timed 500-line test under 200 ms.
Usability	Exact paths, plain messages, fix hints, colour only on terminals.	Mentor review of report clarity.
Portability	Pure Python + PyYAML; `pathlib`; CI on several Python versions.	Green CI matrix 3.10–3.13.

6.3 Maintainability

- **Small, focused modules.** A newcomer can open `redact.py` and understand everything it does without reading the rest of the code.
- **Clear contracts.** Every stage has one main function with typed inputs and outputs (Section 3.5), so changes inside a module do not affect its callers.
- **One-way dependencies.** Shared types sit at the bottom and import nothing; there are no circular imports to untangle.
- **Tests as a safety net.** Golden-file tests catch accidental changes to output; CI blocks any change that breaks tests or lowers coverage.
- **Documentation beside code.** `docs/RULES.md` changes in the same pull request as the rule it describes, enforced by the PR template.

6.4 Scalability

More rules
A new rule is one catalog entry built from existing factories. The engine does not change.

More vendors
Each config family is a separate profile (Intel GPU, NVIDIA, SR-IOV…), selected with `--profile`.

More files
The pipeline runs per file, so a directory is a loop in the CLI. Files are independent, so they could later be validated in parallel.

Bigger files
Work grows linearly with the number of nodes; limits are configurable for unusually large but legitimate configs.

6.5 Future enhancements

Enhancement	How the current architecture supports it	Files touched
External rule files (YAML / JSON)	Rules are already "factory + parameters", so a file entry maps directly to a `Rule` object.	`rules/`
Automatic profile detection	Inspect top-level keys before the engine runs and choose a profile.	`cli.py`, `rules/`
Line numbers on every finding	Swap in a position-aware parser; findings already carry a location field that can be extended.	`loader.py`, `traverser.py`
SARIF output for GitHub code scanning	A new reporter module reading the same `Report` objects.	`reporters/`
pre-commit hook & reusable GitHub Action	Standard exit codes are all these integrations need.	config only
Duplicate-key detection	A custom constructor in the loader reports duplicates as findings.	`loader.py`
Editor / kubectl plugin integration	The pipeline is importable Python, callable without the CLI.	new wrapper

6.6 Risks and mitigations

Risk	Impact	Mitigation
Config schemas differ between vendors	High	Vendor-specific profiles; unknown keys are INFO, never errors; mentor approves sample configs and rules in Week 1.
YAML implicit typing (`yes`, `1.0`, dates)	Medium	Strict type checks with explicit boolean guards; hints suggest quoting values.
Unclear severity for some rules	Medium	Severity is a per-rule field, easy to change after mentor feedback.
Schedule pressure in 4 weeks	Medium	Core features first; stretch goals only after the baseline passes; weekly checkpoints.
Accidental publication of private files	Low	`.gitignore` from the first commit; `git status` before each commit; PR checklist.

6.7 Delivery roadmap

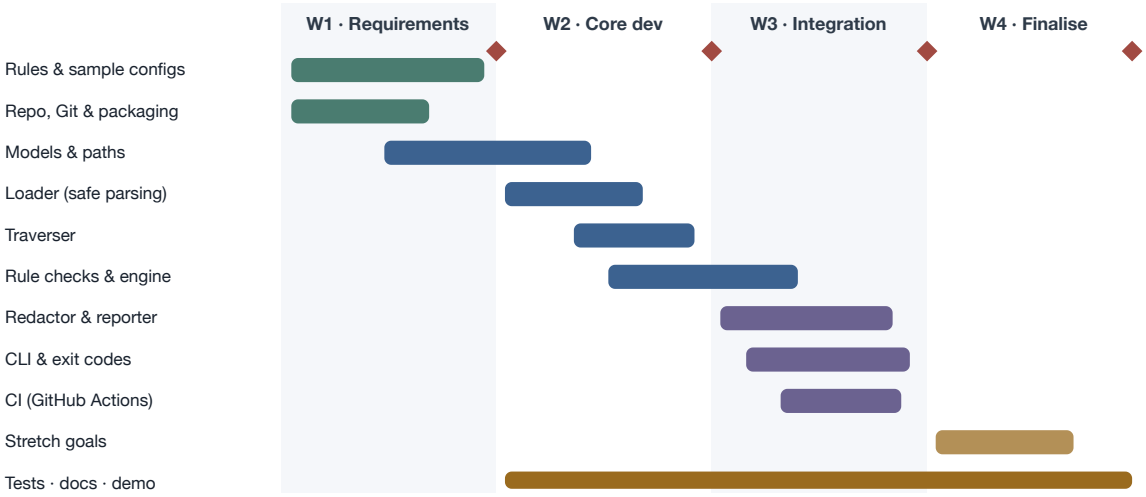


Figure 18. Four-week delivery plan. Red diamonds are mentor checkpoints at the end of each week; testing and documentation run continuously from Week 2.

Week	Deliverables	Mentor checkpoint
W1	Sample configs gathered; rule catalog drafted (keys, types, ranges); repository, `.gitignore`, `pyproject.toml`; `models.py` and `paths.py`.	Approve fixtures, rule definitions and severities.
W2	Safe loader; traverser with limits; check factories; engine; unit tests for each.	Review parsing safety, traversal correctness, first rule tests.
W3	Redactor; text reporter; full CLI with exit codes; GitHub Actions CI.	Review report clarity, redaction safety, CI pipeline.
W4	JSON output and strict mode; coverage ≥ 85%; README, RULES.md; demo; tag `v1.0.0`.	Final evaluation, test evidence, documentation, sign-off.

6.8 Conclusion

The Device Plugin Config Validator addresses a real gap: there is no lightweight, offline tool that checks Kubernetes device plugin configuration before deployment. Existing validators understand Kubernetes manifests, but not the plugin-specific settings that decide how GPUs, FPGAs and network devices behave.

The proposed solution is a small Python command-line tool built as a **pipeline of single-purpose modules**: a safe loader, a depth-first traverser, a rule engine driven by vendor **rule profiles**, a redactor and interchangeable reporters, all coordinated by a thin CLI. Rules are data, secrets are masked before they can reach any output, and hard limits keep the tool safe on hostile input. The technology choices favour the Python standard library, leaving a single runtime dependency.

This architecture keeps the code easy to understand and test today, and leaves clear extension points for tomorrow: new rules, new vendors, new output formats and new integrations can each be added in one place without rewriting the rest. Git and GitHub provide the history, review process and automated testing that hold the project to these standards throughout the four-week schedule and beyond.

A Glossary

Technical terms used in this document, in plain language.

Term	Meaning
Accelerator	Specialised hardware (GPU, FPGA, AI chip) that performs certain computations much faster than a general CPU.
argparse	Python's built-in library for reading command-line options and printing help.
Branch	An independent line of development in Git; a movable label pointing at a commit.
CI (continuous integration)	Automatically building and testing code every time it changes.
CLI	Command-line interface: a program used by typing commands in a terminal.
Commit	A saved snapshot of a project's files in Git, with a message and a unique ID.
ConfigMap	A Kubernetes object that stores configuration data and can be mounted into a pod as a file.
Coverage	The percentage of code lines (or branches) executed by the test suite.
DaemonSet	A Kubernetes workload that runs one copy of a pod on every (or selected) node.
Dataclass	A Python class that mainly stores data; fields are declared with type hints.
Depth-first search (DFS)	A way of walking a tree that follows each branch to the bottom before moving to the next.
Device plugin	A vendor program that advertises special hardware to Kubernetes and allocates it to pods.
Diff	The line-by-line difference between two versions of a file.
Enum	A fixed set of named values, such as ERROR, WARNING and INFO.
Exit code	The number a program returns when it ends; 0 means success.
Factory (function)	A function that creates and returns another function, pre-configured with parameters.
Finding	One reported problem: rule ID, path, severity, message, observed value and hint.
Fixture	A sample input file or prepared object used by tests.
Fork	Your own copy of someone else's GitHub repository, used to propose changes.
Golden file	A saved, approved copy of expected output; tests fail if output changes.
gRPC	A fast remote procedure call protocol; device plugins use it to talk to the kubelet.
Issue	A tracked task, bug or idea in a GitHub repository.
JSON / YAML	Text formats for structured data. YAML is indentation-based and allows comments; JSON uses braces and quotes.
kubelet	The Kubernetes agent on each node that runs and supervises pods.
Merge / merge conflict	Combining branches; a conflict happens when both changed the same lines differently.
Node / pod	A node is a machine in the cluster; a pod is the smallest unit Kubernetes runs (one or more containers).
Profile (rule profile)	The set of rules for one family of configuration files, e.g. one vendor's plugin.
Pull request (PR)	A GitHub proposal to merge one branch into another, with review and CI checks.
Pure function	A function whose result depends only on its inputs and that has no side effects.
Redaction	Replacing sensitive values with a placeholder such as `***REDACTED***`.
Remote / origin	A copy of the repository hosted elsewhere; `origin` is the default name for it.
SafeLoader	PyYAML's loader that only creates plain data types and never executes code.
Severity	How serious a finding is: ERROR (blocks), WARNING (should fix), INFO (for awareness).
Staging area	Git's holding area for changes that will go into the next commit.
Virtual environment	An isolated folder of Python packages for one project.

B Git Command Cheat Sheet

The commands needed for this project, grouped by task.

Set up

Command	Does
`git config --global user.name "Name"`	Set your author name
`git config --global user.email "you@x"`	Set your author email
`git init -b main`	Start a repository
`git clone <url>`	Copy a remote repository
`git remote add origin <url>`	Link to GitHub
`git remote -v`	Show linked remotes

Save work

Command	Does
`git status`	What changed?
`git diff`	Show unstaged changes
`git add <file>`	Stage a file
`git add -p`	Stage parts of files
`git commit -m "msg"`	Commit staged changes
`git restore <file>`	Discard unstaged edits
`git restore --staged <file>`	Unstage a file

Branch and merge

Command	Does
`git branch`	List branches
`git switch -c feature/x`	Create and switch
`git switch main`	Go back to main
`git merge feature/x`	Merge into current
`git branch -d feature/x`	Delete a merged branch

Sync with GitHub

Command	Does
`git push -u origin <branch>`	First push of a branch
`git push`	Push later commits
`git pull`	Fetch and merge
`git fetch`	Fetch without merging

Inspect history

Command	Does
`git log --oneline --graph`	Compact history
`git show <commit>`	One commit's changes
`git blame <file>`	Who changed each line
`git tag v1.0.0`	Mark a release

C References

Documentation consulted for this design.

1. **Kubernetes documentation**: Device Plugins concept guide (architecture, registration and lifecycle of device plugins). kubernetes.io
2. **Intel Device Plugins for Kubernetes**: reference implementations and sample configurations. github.com/intel/intel-device-plugins-for-kubernetes
3. **NVIDIA k8s-device-plugin**: configuration file format and time-slicing options. github.com/NVIDIA/k8s-device-plugin
4. **SR-IOV Network Device Plugin**: `config.json` resource list format. github.com/k8snetworkplumbingwg/sriov-network-device-plugin
5. **PyYAML documentation**: safe loading with `yaml.SafeLoader`. pyyaml.org
6. **Python documentation**: `json`, `argparse`, `dataclasses`, `enum`, `pathlib`. docs.python.org
7. **pytest** and **coverage.py** documentation: fixtures, parametrisation, coverage gates. docs.pytest.org · coverage.readthedocs.io
8. **Pro Git** (Chacon & Straub) and the Git reference manual. git-scm.com
9. **GitHub Docs**: repositories, issues, pull requests, Actions, branch protection. docs.github.com
10. **G401 Project Proposal (PRD)**: Kubernetes Device Plugin Config Validator.